Jenkins, CI/CD & GitHub Integration

Week 5 Report • DevOps Internship • Davine Technologies

This report documents my hands-on work with Jenkins and CI/CD during Week 5 of the DevOps internship. I installed Jenkins, connected it to GitHub with securely stored credentials, built both a Freestyle Project and a Declarative Pipeline defined by a Jenkinsfile, and configured a real GitHub Webhook (via an ngrok tunnel) so that pushes to my repository automatically trigger builds. Each section below explains a core concept and how I applied it this week.

1. CI/CD

CI/CD stands for Continuous Integration and Continuous Delivery/Deployment. **Continuous Integration** means that every time a developer pushes code, it is automatically built and tested, catching bugs immediately instead of days later. **Continuous Delivery/Deployment** means that code which passes CI is automatically prepared, and optionally deployed, for release. This week's task focused on the CI side: building a pipeline that automatically validates every code change pushed to GitHub, without a human manually running the build and test steps each time.

2. Jenkins Architecture

Jenkins is an automation server that sits between a code repository and the build/test/deploy process. It follows a simple architecture:

• **Jenkins Server (daemon)** — runs as a background service (I installed it via apt and manage it with `systemctl`), listening on port 8080.
• **Jobs/Projects** — defined units of work, such as a Freestyle Project or a Pipeline.
• **Plugins** — Jenkins is extensible; the Git, GitHub, and Pipeline plugins (installed via "suggested plugins" during setup) provide the SCM integration and pipeline syntax I used.
• **Executors** — the workers that actually run builds on the Jenkins node.
• **Credentials Store** — securely stores secrets Jenkins needs, such as my GitHub Personal Access Token.

One important lesson from setup: Jenkins requires a specific Java version to run — my installation initially failed because it was running under Java 17, while Jenkins 2.568.x requires Java 21 or newer. Installing OpenJDK 21 and setting it as the default resolved this.

3. Freestyle Jobs

A Freestyle Project is Jenkins' original, UI-based job type. Instead of writing pipeline code, you configure everything through forms: which Git repository to pull from, which credentials to use, what triggers a build, and a sequence of build steps (in my case, an "Execute shell" step). I created a Freestyle job (`week5-freestyle-build`) that:

- Pulled source code from my GitHub repository using the stored credentials
- Ran node --version to confirm the environment, then executed the app (node app.js)

● Ran a test script (node app.test.js) and reported pass/fail

This job is simple to set up but not version-controlled — its configuration lives only inside Jenkins itself, which is why real-world teams prefer Pipelines defined as code.

4. Declarative Pipelines

A Declarative Pipeline defines an entire build process as structured code, using a fixed, readable syntax (`pipeline { agent ... stages { ... } }`). Unlike a Freestyle job, this definition lives in the repository itself as a Jenkinsfile, so it is versioned, reviewable, and portable between Jenkins instances. I created a Pipeline job (`week5-declarative-pipeline`) configured with "Pipeline script from SCM", pointing at the Jenkinsfile in my repository.

5. Jenkinsfile

The Jenkinsfile I wrote and committed to my repository:

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                echo 'Checking out code from GitHub...'
                checkout scm
            }
        }

        stage('Build') {
            steps {
                echo 'Building the application...'
                sh 'node --version'
                sh 'node app.js'
            }
        }

        stage('Test') {
            steps {
                echo 'Running tests...'
                sh 'node app.test.js'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully!'
        }
        failure {
            echo 'Pipeline failed. Check the console output above.'
        }
```

```
    }
}
```

`agent any` tells Jenkins to run on any available executor. Each `stage` block is a distinct, named phase of the pipeline, and the `post` block runs cleanup or notification logic depending on whether the pipeline succeeded or failed.

6. Pipeline Stages

Stages break a pipeline into logical phases, each shown as its own block in Jenkins' visual Stage View, making it easy to see exactly where a build succeeded or failed. My pipeline has three stages:

- Checkout — pulls the latest code from GitHub (explicitly, via checkout scm, for clarity, though Jenkins also performs this automatically for SCM-backed pipelines)
- Build — verifies the Node.js environment and runs the application
- Test — runs the test script; if a test throws an error, this stage (and the whole pipeline) is marked as failed

Running the pipeline produced a Stage View with Checkout SCM, Checkout, Build, Test, and Post Actions all shown as separate green blocks, confirming each phase executed and passed independently.

7. Jenkins Credentials

Rather than hardcoding a GitHub username and password/token into job configuration, Jenkins provides a Credentials Store. I generated a GitHub Personal Access Token (scoped to `repo` access only) and added it to Jenkins under Manage Jenkins → Credentials, as a "Username with password" credential with the ID `github-credentials`. Both my Freestyle job and my Pipeline job reference this credential ID to authenticate against GitHub when cloning — the actual token value is never exposed in job configuration or logs.

8. GitHub Webhooks

A webhook lets GitHub notify Jenkins the instant code is pushed, instead of Jenkins having to repeatedly poll GitHub for changes. Since my Jenkins instance runs on localhost inside a private VM, I used **ngrok** to create a temporary public URL tunnelling to `localhost:8080`, then configured a webhook on my GitHub repository pointing at `https://<ngrok-url>/github-webhook/` with the "push" event enabled.

Troubleshooting the webhook (a genuine debugging exercise):

- First failure: HTTP 405 Method Not Allowed on every delivery. Diagnosis: Jenkins' CSRF crumb validation was rejecting requests arriving through the ngrok reverse proxy, because the Host header didn't match what Jenkins expected.
- Fix: edited Jenkins' config.xml to set excludeClientIPFromCrumb=true on the crumb issuer (equivalent to the UI's "Enable proxy compatibility" option) and restarted Jenkins. A manual test request then succeeded with 200 OK.
- Second failure: real GitHub deliveries still failed, now with HTTP 302 Found, redirecting to the exact same path. Inspecting the actual request URL GitHub sent revealed the payload URL was missing a

trailing slash (/github-webhook instead of /github-webhook/) — Jenkins was redirecting to the correct path, but GitHub's webhook delivery system does not follow redirects on POST requests, so every delivery was marked failed.

- Resolution: added the trailing slash to the webhook's Payload URL in GitHub's repository settings. The very next push produced a 200 OK delivery and triggered both the Freestyle job and the Pipeline job automatically, with zero manual "Build Now" clicks.

This turned out to be one of the most instructive parts of the week: reading Jenkins' access logs (which show the exact request path and response code for every delivery) and comparing a working manual curl request against a failing real GitHub request was what ultimately isolated the root cause, rather than guessing at configuration changes.

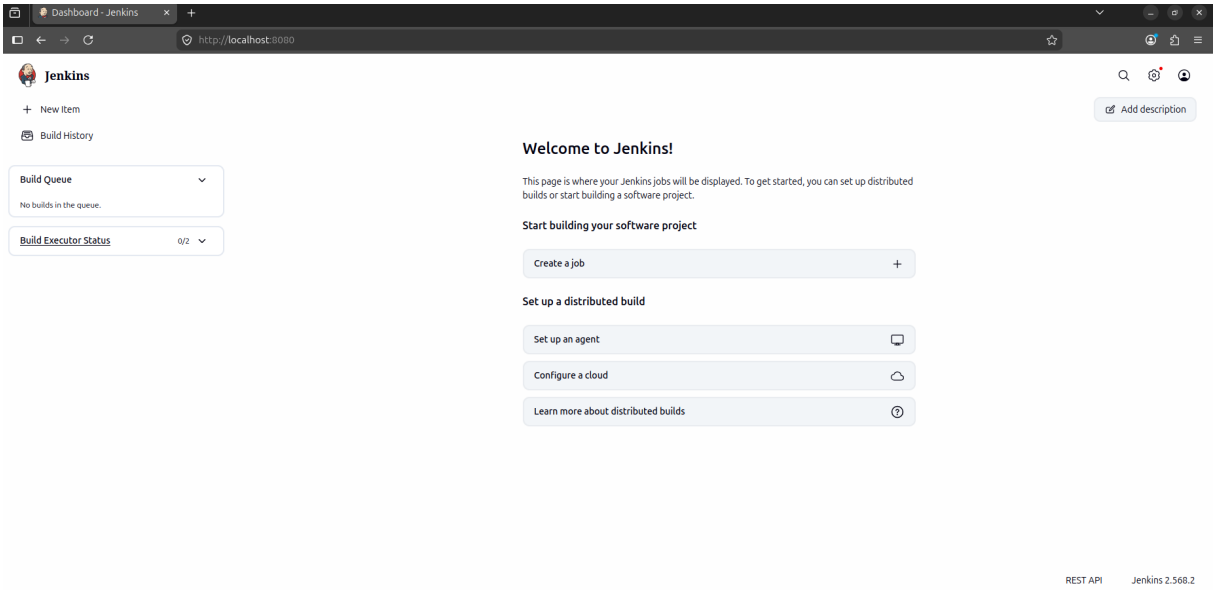
Summary

- Installed and configured Jenkins on Ubuntu, resolving a Java version compatibility issue along the way.
- Connected Jenkins to GitHub using a securely stored Personal Access Token credential.
- Built a Freestyle Project that clones a repository, builds, and tests a Node.js application.
- Wrote a Jenkinsfile defining a Declarative Pipeline with Checkout, Build, and Test stages, executed successfully with a full green Stage View.
- Exposed Jenkins publicly via an ngrok tunnel and configured a real GitHub Webhook.
- Diagnosed and fixed two distinct webhook failures (a CSRF/proxy issue, then a trailing-slash redirect issue) by reading Jenkins' request logs and comparing successful vs. failing deliveries.
- Confirmed the full automated CI loop: a git push now triggers both jobs in Jenkins with no manual intervention.

Appendix: Screenshots

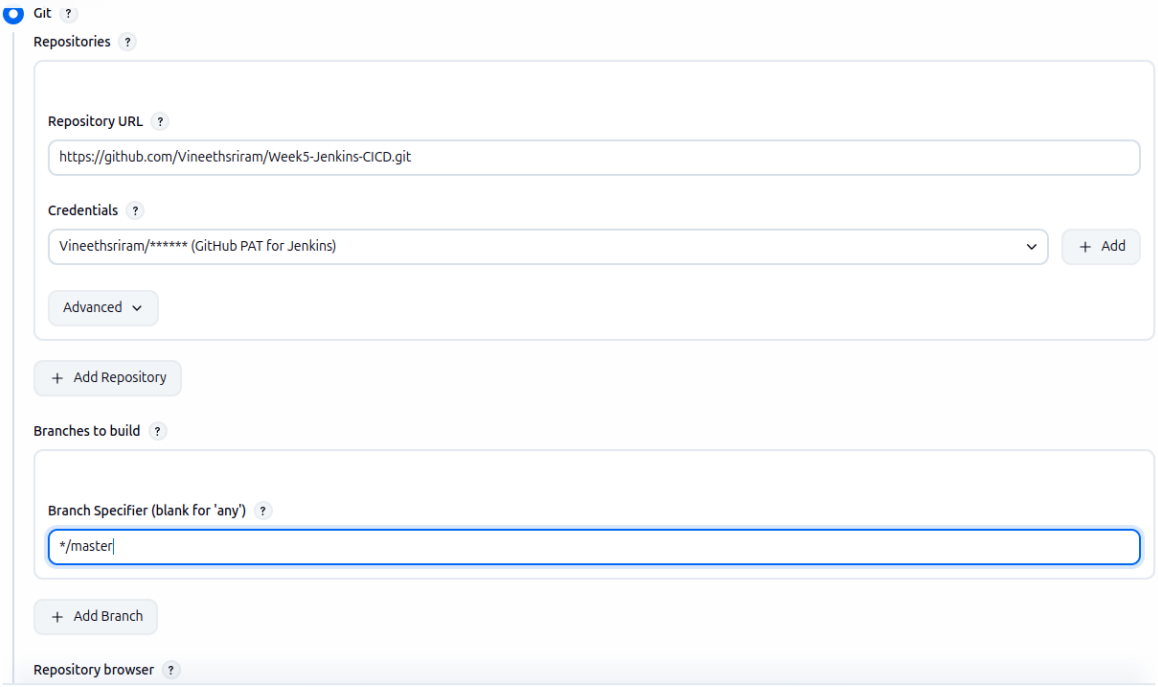
Supporting evidence for each required deliverable

Jenkins Dashboard



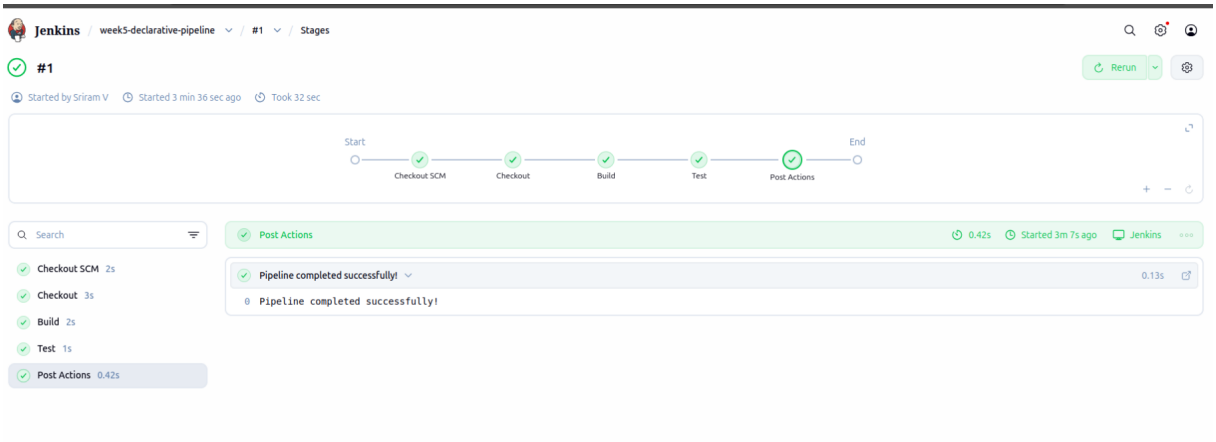
The Jenkins dashboard after a successful installation and setup wizard, running version 2.568.2.

GitHub Integration



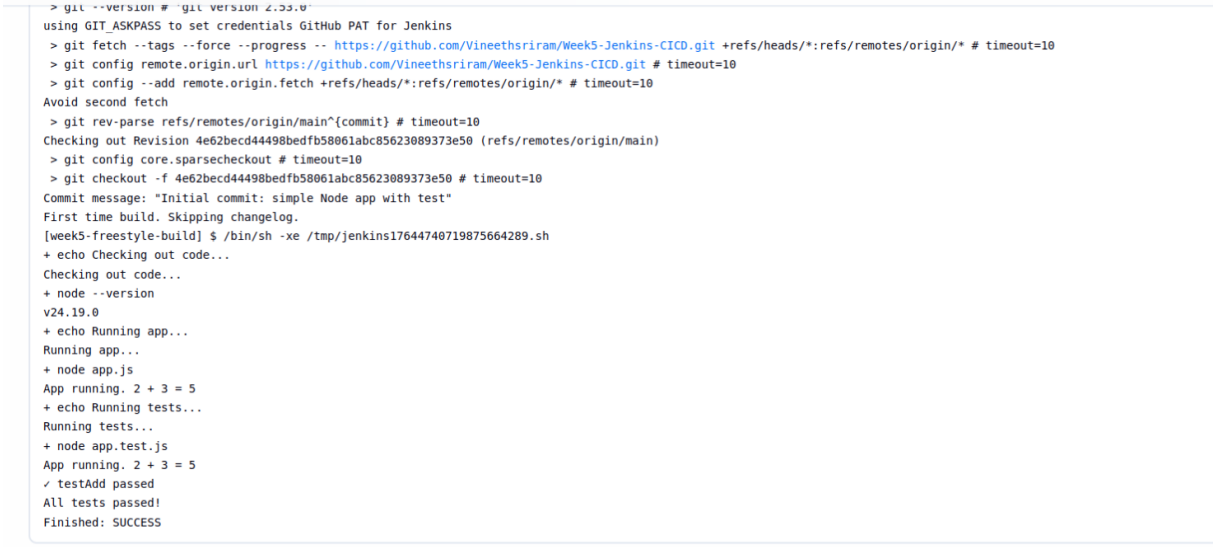
Jenkins job configuration showing the Git repository URL and stored credentials used to pull code from GitHub.

Pipeline Execution



The Declarative Pipeline's Stage View, showing Checkout, Build, and Test stages all completed successfully.

Build Logs



```
 > git --version # 'git version 2.53.0'
using GIT_ASKPASS to set credentials GitHub PAT for Jenkins
 > git fetch --tags --force --progress -- https://github.com/Vineethsriram/Week5-Jenkins-CICD.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/Vineethsriram/Week5-Jenkins-CICD.git # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 4e62becd44498bedfb58061abc85623089373e50 (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 4e62becd44498bedfb58061abc85623089373e50 # timeout=10
Commit message: "Initial commit: simple Node app with test"
First time build. Skipping changelog.
[week5-freestyle-build] $ /bin/sh -xe /tmp/jenkins17644740719875664289.sh
+ echo Checking out code...
Checking out code...
+ node --version
v24.19.0
+ echo Running app...
Running app...
+ node app.js
App running. 2 + 3 = 5
+ echo Running tests...
Running tests...
+ node app.test.js
App running. 2 + 3 = 5
✓ testAdd passed
All tests passed!
Finished: SUCCESS
```

Console output of a successful build, showing the checkout, build, and test steps executing with a final SUCCESS result.